Project - Low Level Designon

CI/CD for Vue.js Technology App

Course Name: Devops Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Pranjal Mahwar	EN22CS301729
2.	Prateeksha Shukla	EN22CS301740
3.	Parthiv Prajapat	EN22EL301039
4.	Raj Patel	EN22CS301781
5.	Rishabh Dev Rajput	EN22CS17803

Group Name:06D7

Project Number:D0-29

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

This document provides a detailed overview of the design and implementation of a CI/CD pipeline for a Vue.js application using modern DevOps practices. The system leverages cloud infrastructure provided by Amazon Web Services, specifically Amazon EC2 instances, to ensure scalability, reliability, and ease of deployment.

The project integrates tools such as Git, Docker, GitHub Actions/Jenkins, and Kubernetes to automate the software delivery lifecycle, from code integration to deployment.

1.1 Scope of the Document

This document covers:

- Design and setup of CI/CD pipeline for a Vue.js application
- Configuration of cloud infrastructure using AWS EC2 instances
- Integration of version control (Git) with CI/CD tools (GitHub Actions/Jenkins)
- Containerization using Docker
- Deployment using Kubernetes (K8s)
- Automation of build, test, and deployment processes

It provides both conceptual understanding and practical implementation details required for the project.

1.2 Intended Audience

This document is intended for:

- Students and instructors involved in the DevOps project
- Developers working on Vue.js applications
- DevOps engineers and beginners learning CI/CD pipelines
- System administrators managing cloud-based deployments

1.3 System Overview

The system is designed as a fully automated CI/CD pipeline deployed on AWS infrastructure. The workflow is as follows:

1. **Code Development**
   Developers write and push code to a Git repository (e.g., GitHub).
2. **Continuous Integration (CI)**
   - Triggered on every commit
   - Automated build and unit testing using GitHub Actions or Jenkins
   - Ensures code quality and early bug detection
3. **Containerization**
   - Application is packaged into Docker containers
   - Images are stored in a container registry
4. **Cloud Infrastructure (AWS EC2)**
   - EC2 instances act as servers for hosting the application
   - Provides scalable and flexible compute resources
5. **Continuous Deployment (CD)**
   - On successful build, the application is deployed automatically
   - Kubernetes manages container orchestration and scaling
6. **Monitoring & Reliability**
   - Ensures high availability and fast deployment cycles
   - Supports enterprise-level application performance

This architecture ensures a streamlined development lifecycle with minimal manual intervention, faster releases, and improved software quality.

2.1 Application Design

The application design focuses on how the Vue.js application is structured, built, and deployed within the CI/CD pipeline.

1. Frontend Application (Vue.js)

- The application is developed using Vue.js framework
- Follows component-based architecture
- Organized into:
  - Components (UI elements)
  - Views (pages)
  - Services (API calls)
  - Assets (CSS, images)

---

2. Version Control Integration

- Source code is stored in GitHub repository
- Developers push changes to branches (main/develop)
- Each push triggers the CI/CD pipeline automatically

---

3. Build Process

- CI tool (GitHub Actions/Jenkins) installs dependencies
- Runs build command:

```
npm install
npm run build
```

- Generates optimized production-ready files

---

4. Containerization using Docker

- Application is packaged into a Docker image
- A Dockerfile defines:
  - Base image (Node/Nginx)
  - Build steps
  - Deployment configuration
- Ensures consistency across environments

5. Deployment Architecture

- Docker container is deployed on Amazon EC2
- Kubernetes manages:
  - Pod creation
  - Scaling
  - Load balancing

6. CI/CD Workflow Integration

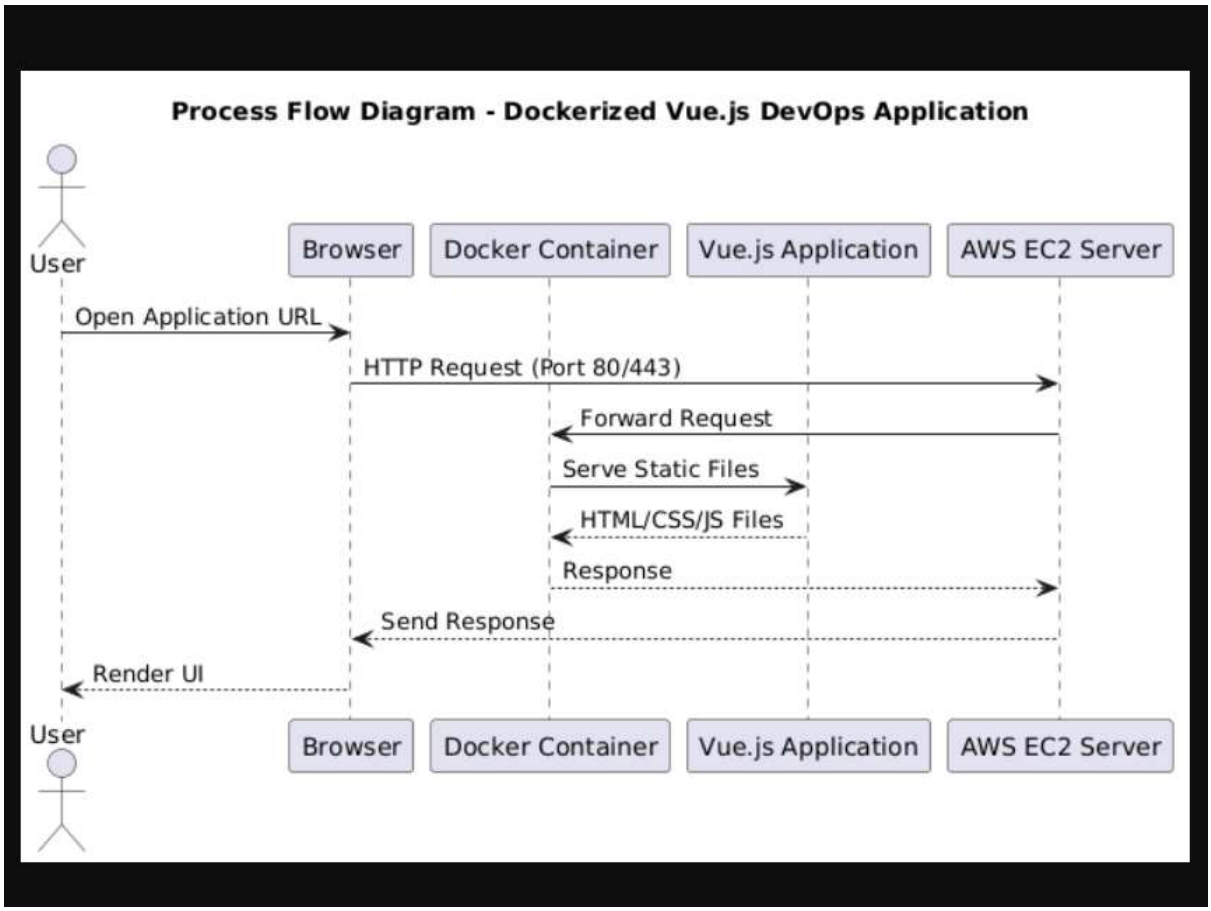
- **CI Phase:**
  - Code push triggers pipeline
  - Build + test execution
- **CD Phase:**
  - Docker image creation
  - Deployment to EC2 via Kubernetes

7. Key Features of Application Design

- Modular and reusable components
- Automated build and deployment
- Scalable cloud-based hosting
- High availability and reliability

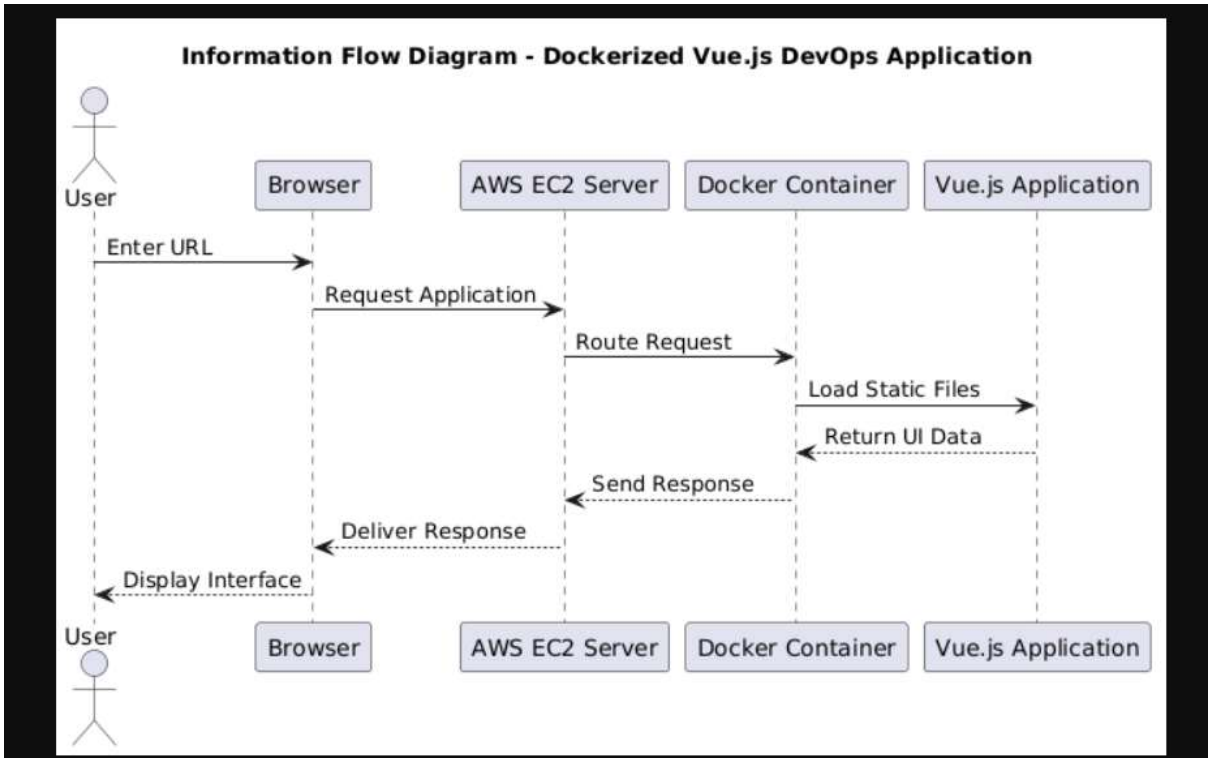
2.2 Process Flow Description:

The process flow represents the sequence of operations performed when a user accesses the application.



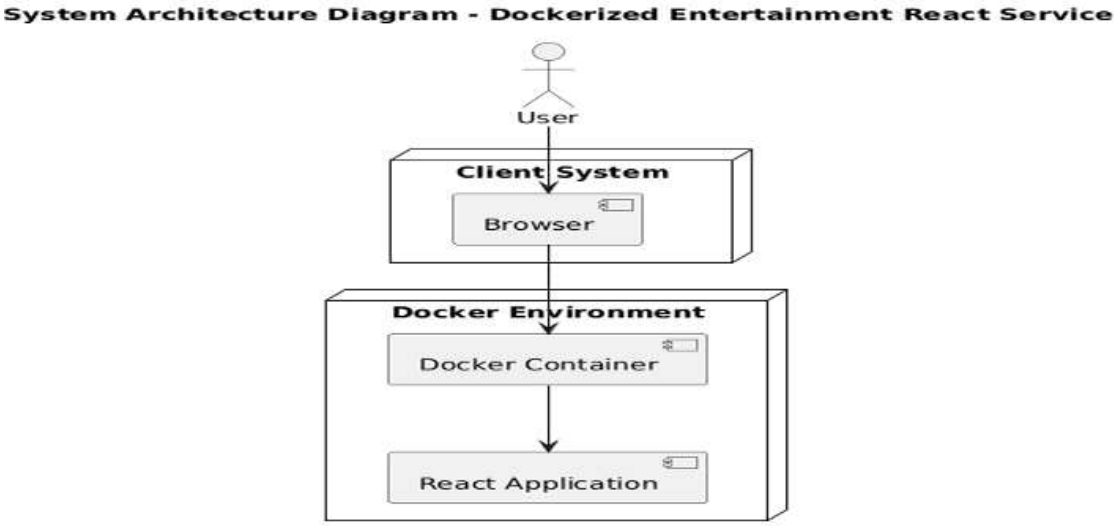
2.3 Information Flow :

The information flow describes how data is transferred between different components of the system.



2.4 System Architecture :

The architecture diagram provides a high-level representation of system components and their relationships.



2.5 Key Design Considerations

The system design is developed by considering several important factors to ensure efficiency, scalability, and reliability of the application deployed on Amazon Web Services using Amazon EC2.

1. Scalability

- The system is designed to scale horizontally using containerization and orchestration tools like Kubernetes
- AWS EC2 instances can be resized or replicated based on demand

2. Reliability

- Ensures continuous availability of the application
- Use of container-based deployment reduces failure impact
- Fault tolerance is achieved through multiple instances

3. Performance

- Optimized build process for Vue.js application
- Fast response time using lightweight containers
- Efficient resource utilization on EC2

4. Security

- Secure access using SSH keys for EC2
- Environment variables and secrets managed securely in CI/CD pipelines
- HTTPS support for secure communication

5. Automation

- Full CI/CD pipeline using GitHub Actions or Jenkins
- Automated build, test, and deployment processes
- Reduces manual errors and speeds up delivery

6. Maintainability

- Modular application structure (Vue components)
- Easy updates and rollback using Docker images

2.7 API Catalogue Current Implementation:

No APIs available (Static frontend application for now)

3. References

The following resources were used for designing and implementing the project:

1. Amazon Web Services Documentation – https://docs.aws.amazon.com/
2. Docker Documentation – https://docs.docker.com/
3. Kubernetes Documentation – https://kubernetes.io/docs/
4. GitHub Actions Documentation – https://docs.github.com/actions
5. Jenkins Documentation – https://www.jenkins.io/doc/
6. Vue.js Documentation – https://vuejs.org/guide/introduction.html